
Session 18 - AIML Optional Task

Github Folder Link : https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks_assignments/tree/main/Session18

➤ **Insurance Dataset**

Link: <https://www.kaggle.com/datasets/mirichoi0218/insurance>

✧ Github Link : https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks_assignments/blob/main/Session18/OptionalTask%20Session%2018%20-%20Insurance.ipynb

➤ **Iris Dataset - Dataset used for Session 16's Optional Task**

Link: <https://www.kaggle.com/datasets/uciml/iris>

✧ Github Link : https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks_assignments/blob/main/Session18/OptionalTask%20Session%2018%20-%20Iris.ipynb

➤ **WineQT Dataset - Another new dataset from kaggle**

Link: <https://www.kaggle.com/datasets/yasserh/wine-quality-dataset>

✧ Github Link : https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks_assignments/blob/main/Session18/OptionalTask%20Session%2018%20-%20WineQT.ipynb

Insurance Dataset - Information

• Dataset Description

- The Insurance Dataset contains information about individuals and their medical insurance expenses.
- It includes personal and health-related details that influence insurance costs.
- The dataset is commonly used for analyzing factors affecting medical expenses and building regression models.

• Features

- Age
- Sex
- BMI
- Children
- Smoker
- Region

• Target Variable

- Expenses

• Machine Learning Type

- Regression

• Purpose

- To analyze the relationship between personal factors and insurance expenses.
- To predict medical insurance costs using input features.

Program Output:

1. Import Libraries

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

plt.style.use("ggplot")
sns.set_theme(style="whitegrid")
```

2. Data Loading

```
[2]: # Load the dataset
df = pd.read_csv("insurance.csv")

# Display first 10 rows
print("First 10 Rows:")
print(df.head(10))

# Display last 5 rows
print("\nLast 5 Rows:")
print(df.tail())

# Display shape
print("\nDataset Shape:")
print(df.shape)

# Display data types
print("\nData Types:")
print(df.dtypes)

# Display dataset information
print("\nDataset Information:")
df.info()
```

```

First 10 Rows:
  age  sex  bmi  children  smoker  region  expenses
0  19  female  27.9      0     yes  southwest  16884.92
1  18   male  33.8      1     no   southeast  1725.55
2  28   male  33.0      3     no   southeast  4449.46
3  33   male  22.7      0     no  northwest  21984.47
4  32   male  28.9      0     no  northwest  3866.86
5  31  female  25.7      0     no   southeast  3756.62
6  46  female  33.4      1     no   southeast  8240.59
7  37  female  27.7      3     no  northwest  7281.51
8  37   male  29.8      2     no  northeast  6406.41
9  60  female  25.8      0     no  northwest  28923.14

Last 5 Rows:
  age  sex  bmi  children  smoker  region  expenses
1333  50  male  31.0      3     no  northwest  10600.55
1334  18  female  31.9      0     no  northeast  2205.98
1335  18  female  36.9      0     no  southeast  1629.83
1336  21  female  25.8      0     no  southwest  2007.95
1337  61  female  29.1      0     yes  northwest  29141.36

Dataset Shape:
(1338, 7)

Data Types:
age      int64
sex      object
bmi      float64
children int64
smoker   object
region   object
expenses float64
dtype: object

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   age         1338 non-null   int64
1   sex         1338 non-null   object
2   bmi         1338 non-null   float64
3   children    1338 non-null   int64
4   smoker      1338 non-null   object
5   region      1338 non-null   object
6   expenses    1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB

```

3. Missing Values

```

[3]: # Check missing values
print("Missing Values:")
print(df.isnull().sum())

# Check duplicate rows
duplicate_count = df.duplicated().sum()

print("\nDuplicate Rows:", duplicate_count)

# Remove duplicate rows
df = df.drop_duplicates()

print("\nDuplicate Rows After Removal:")
print(df.duplicated().sum())

```

```

Missing Values:
age      0
sex      0
bmi      0
children 0
smoker   0
region   0
expenses 0
dtype: int64

```

```

Duplicate Rows: 1

```

```

Duplicate Rows After Removal:
0

```

4. Statistical Summary

```
[4]: # Display statistical summary
print("Statistical Summary:")
print(df.describe())

# Statistics for important numeric columns
columns = ["age", "bmi", "children", "expenses"]

for col in columns:
    print(f"\n{col.upper()} Statistics")
    print("Minimum :", df[col].min())
    print("Maximum :", df[col].max())
    print("Mean      :", df[col].mean())
    print("Median     :", df[col].median())
```

```
Statistical Summary:
      age      bmi  children  expenses
count 1337.000000 1337.000000 1337.000000 1337.000000
mean   39.222139  30.665520   1.095737 13279.121638
std    14.044333   6.100664   1.205571 12110.359657
min    18.000000  16.000000   0.000000 1121.870000
25%    27.000000  26.300000   0.000000  4746.340000
50%    39.000000  30.400000   1.000000  9386.160000
75%    51.000000  34.700000   2.000000 16657.720000
max     64.000000  53.100000   5.000000 63770.430000
```

AGE Statistics

```
Minimum : 18
Maximum : 64
Mean     : 39.222139117427076
Median   : 39.0
```

BMI Statistics

```
Minimum : 16.0
Maximum : 53.1
Mean     : 30.665519820493646
Median   : 30.4
```

CHILDREN Statistics

```
Minimum : 0
Maximum : 5
Mean     : 1.0957367240089753
Median   : 1.0
```

EXPENSES Statistics

```
Minimum : 1121.87
Maximum : 63770.43
Mean     : 13279.121637995511
Median   : 9386.16
```

5. Histogram Analysis

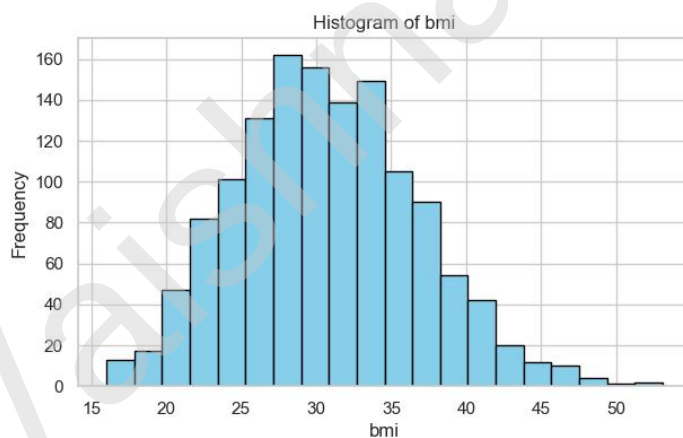
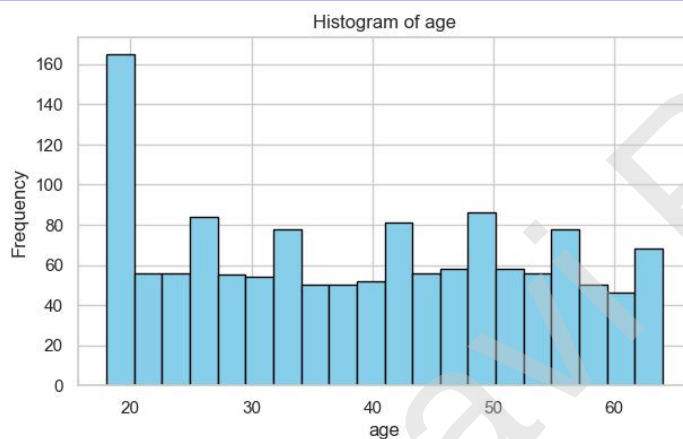
```
[5]: # List of numeric columns
numeric_columns = ["age", "bmi", "children", "expenses"]

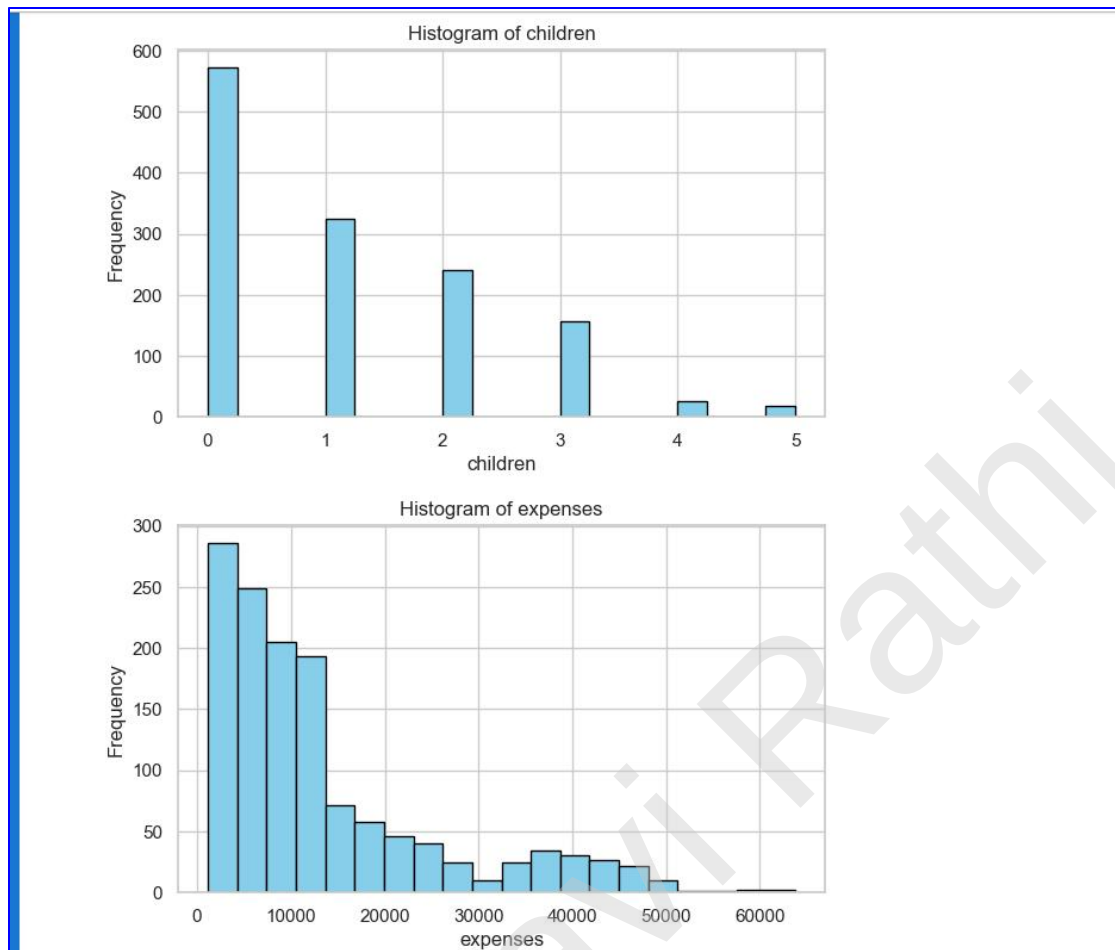
# Plot histograms
for col in numeric_columns:
    plt.figure(figsize=(7, 4))

    plt.hist(df[col], bins=20, edgecolor="black", color="skyblue")

    plt.title(f"Histogram of {col}")
    plt.xlabel(col)
    plt.ylabel("Frequency")

    plt.show()
```





6. Count Plot Analysis

```
[6]: # List of categorical columns
categorical_columns = ["sex", "smoker", "region"]

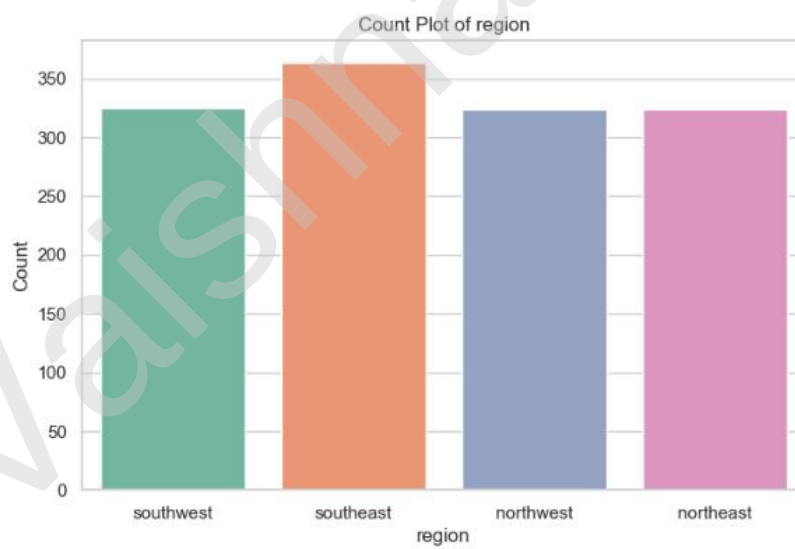
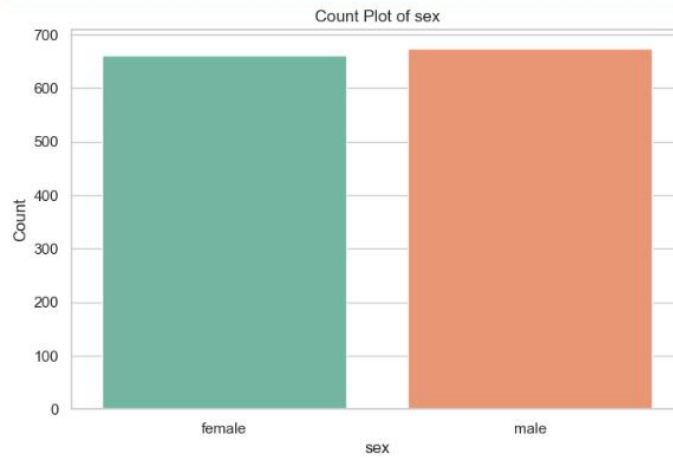
# Plot count plots
for col in categorical_columns:

    plt.figure(figsize=(8, 5))

    sns.countplot(
        data=df,
        x=col,
        hue=col,
        palette="Set2",
        legend=False
    )

    plt.title(f"Count Plot of {col}")
    plt.xlabel(col)
    plt.ylabel("Count")

    plt.show()
```



7. Correlation Heatmap

```
[7]: # Select numeric columns
numeric_df = df.select_dtypes(include="number")

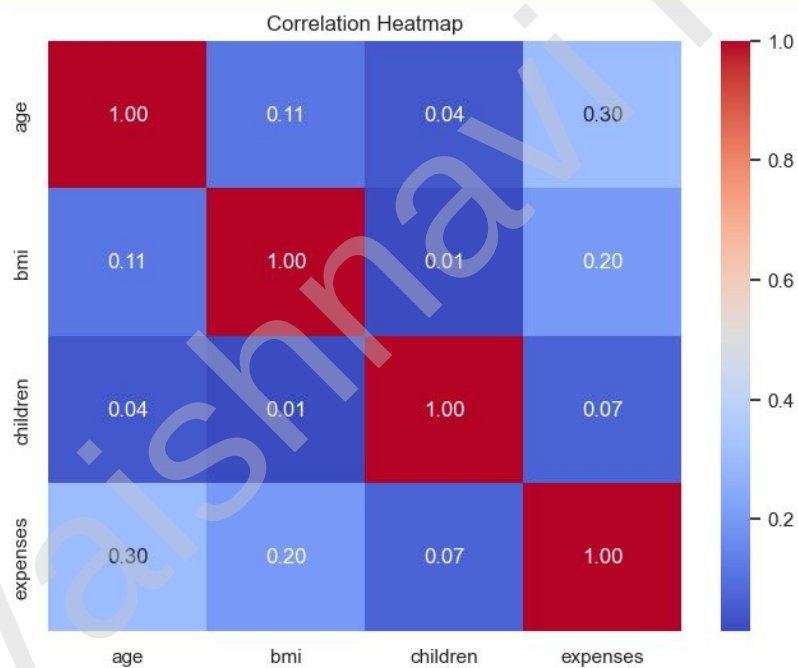
# Create correlation matrix
corr_matrix = numeric_df.corr()

# Plot heatmap
plt.figure(figsize=(8, 6))

sns.heatmap(
    corr_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Heatmap")
plt.show()

# Display correlation with expenses
print("Correlation with Expenses:")
print(corr_matrix["expenses"].sort_values(ascending=False))
```



```
Correlation with Expenses:
expenses    1.000000
age         0.298308
bmi         0.198637
children    0.067389
Name: expenses, dtype: float64
```

8. Feature Identification

```
[8]: # Independent Features
X = df.drop("expenses", axis=1)

# Dependent Feature
y = df["expenses"]

print("Independent Features:")
print(X.columns.tolist())

print("\nDependent Feature:")
print(y.name)

Independent Features:
['age', 'sex', 'bmi', 'children', 'smoker', 'region']

Dependent Feature:
expenses
```

9. One Hot Encoding

```
[9]: # Display original categorical columns
print("Before One Hot Encoding:")
print(df[["sex", "smoker"]].head())

# Apply One Hot Encoding
encoded_df = pd.get_dummies(
    df,
    columns=["sex", "smoker", "region"],
    drop_first=True
)

# Display encoded columns
print("\nAfter One Hot Encoding:")
print(encoded_df.head())

print("\nShape Before Encoding:", df.shape)
print("Shape After Encoding:", encoded_df.shape)
```

Before One Hot Encoding:

	sex	smoker
0	female	yes
1	male	no
2	male	no
3	male	no
4	male	no

After One Hot Encoding:

	age	bmi	children	expenses	sex_male	smoker_yes	region_northwest	\
0	19	27.9	0	16884.92	False	True	False	
1	18	33.8	1	1725.55	True	False	False	
2	28	33.0	3	4449.46	True	False	False	
3	33	22.7	0	21984.47	True	False	True	
4	32	28.9	0	3866.86	True	False	True	

	region_southeast	region_southwest
0	False	True
1	True	False
2	True	False
3	False	False
4	False	False

Shape Before Encoding: (1337, 7)

Shape After Encoding: (1337, 9)

10. Feature Scaling

```
[10]: from sklearn.preprocessing import StandardScaler

# Select numeric independent features
numeric_features = ["age", "bmi", "children"]

# Create StandardScaler object
scaler = StandardScaler()

# Apply scaling
scaled_data = scaler.fit_transform(df[numeric_features])

# Convert to DataFrame
scaled_df = pd.DataFrame(
    scaled_data,
    columns=numeric_features
)

# Display first 5 rows
print("First 5 Rows of Scaled Data:")
print(scaled_df.head())
```

First 5 Rows of Scaled Data:

	age	bmi	children
0	-1.440418	-0.453484	-0.909234
1	-1.511647	0.513986	-0.079442
2	-0.799350	0.382803	1.580143
3	-0.443201	-1.306169	-0.909234
4	-0.514431	-0.289506	-0.909234

11. Preprocessing Pipeline

```
[11]: from sklearn.preprocessing import StandardScaler

# Remove duplicate rows
df = df.drop_duplicates()

# Define input and output features
X = df.drop("expenses", axis=1)
y = df["expenses"]

# Apply One Hot Encoding
X = pd.get_dummies(
    X,
    columns=["sex", "smoker", "region"],
    drop_first=True
)

# Select numeric columns for scaling
numeric_features = ["age", "bmi", "children"]

# Apply Standard Scaling
scaler = StandardScaler()
X[numeric_features] = scaler.fit_transform(X[numeric_features])

# Display preprocessed data
print("Preprocessed Dataset:")
print(X.head())

print("\nTarget Variable:")
print(y.head())
```

```
Preprocessed Dataset:
   age      bmi  children  sex_male  smoker_yes  region_northwest \
0 -1.440418 -0.453484 -0.909234    False        True             False
1 -1.511647  0.513986 -0.079442     True        False             False
2 -0.799350  0.382803  1.580143     True        False             False
3 -0.443201 -1.306169 -0.909234     True        False              True
4 -0.514431 -0.289506 -0.909234     True        False              True

   region_southeast  region_southwest
0             False                True
1              True                False
2              True                False
3             False                False
4             False                False

Target Variable:
0    16884.92
1     1725.55
2     4449.46
3    21984.47
4     3866.86
Name: expenses, dtype: float64
```

12. Dataset Summary

▼

Summary

- **Data Exploration**
 - Loaded and explored the dataset.
 - Checked the shape, columns, and data types.
- **Data Cleaning**
 - Identified missing values and duplicate records.
 - Cleaned the dataset for analysis.
- **Data Analysis**
 - Generated summary statistics.
 - Created histograms, count plots, and a correlation heatmap.
- **Preprocessing**
 - Identified input and target features.
 - Applied One Hot Encoding and Standard Scaling.
- **Key Learning**
 - Learned the complete data preprocessing workflow.
 - Prepared the dataset for a regression machine learning model.

Iris Dataset - Information

• Dataset Description

- The Iris Dataset contains measurements of Iris flowers belonging to three different species: Setosa, Versicolor, and Virginica.
- It is a well-known dataset used for understanding classification problems in machine learning.
- The dataset helps identify patterns between flower measurements and species.

• Features

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

• Target Variable

- Species

• Machine Learning Type

- Classification

• Purpose

- To classify Iris flower species based on their physical measurements.
- To understand the relationship between flower features and categories.

Program Output:

1. Import Libraries

```
[2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder, StandardScaler

plt.style.use("ggplot")
sns.set_theme(style="whitegrid")
```

2. Data Loading

```
[3]: # Load the dataset
df = pd.read_csv("Iris.csv")

# Display first 10 rows
print("First 10 Rows:")
print(df.head(10))

# Display last 5 rows
print("\nLast 5 Rows:")
print(df.tail())

# Display dataset shape
print("\nDataset Shape:")
print(df.shape)

# Display column names
print("\nColumn Names:")
print(df.columns.tolist())

# Display data types
print("\nData Types:")
print(df.dtypes)

# Display dataset information
print("\nDataset Information:")
df.info()
```

```

First 10 Rows:
  Id  SepalLengthCm  SepalwidthCm  PetalLengthCm  PetalwidthCm  Species
0   1             5.1           3.5           1.4           0.2  Iris-setosa
1   2             4.9           3.0           1.4           0.2  Iris-setosa
2   3             4.7           3.2           1.3           0.2  Iris-setosa
3   4             4.6           3.1           1.5           0.2  Iris-setosa
4   5             5.0           3.6           1.4           0.2  Iris-setosa
5   6             5.4           3.9           1.7           0.4  Iris-setosa
6   7             4.6           3.4           1.4           0.3  Iris-setosa
7   8             5.0           3.4           1.5           0.2  Iris-setosa
8   9             4.4           2.9           1.4           0.2  Iris-setosa
9  10             4.9           3.1           1.5           0.1  Iris-setosa

Last 5 Rows:
  Id  SepalLengthCm  SepalwidthCm  PetalLengthCm  PetalwidthCm  \
145 146           6.7           3.0           5.2           2.3
146 147           6.3           2.5           5.0           1.9
147 148           6.5           3.0           5.2           2.0
148 149           6.2           3.4           5.4           2.3
149 150           5.9           3.0           5.1           1.8

Species
145 Iris-virginica
146 Iris-virginica
147 Iris-virginica
148 Iris-virginica
149 Iris-virginica

Dataset Shape:
(150, 6)

Column Names:
['Id', 'SepalLengthCm', 'SepalwidthCm', 'PetalLengthCm', 'PetalwidthCm', 'Species']

Data Types:
Id           int64
SepalLengthCm  float64
SepalwidthCm  float64
PetalLengthCm  float64
PetalwidthCm  float64
Species       object
dtype: object

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  --
0   Id               150 non-null   int64
1   SepalLengthCm    150 non-null   float64
2   SepalwidthCm     150 non-null   float64
3   PetalLengthCm    150 non-null   float64
4   PetalwidthCm     150 non-null   float64
5   Species          150 non-null   object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB

```

3. Missing Values

```

[4]: # Check missing values
print("Missing Values:")
print(df.isnull().sum())

# Check duplicate rows
duplicate_count = df.duplicated().sum()

print("\nDuplicate Rows:", duplicate_count)

# Remove duplicate rows
df = df.drop_duplicates()

print("\nDuplicate Rows After Removal:")
print(df.duplicated().sum())

Missing Values:
Id           0
SepalLengthCm  0
SepalwidthCm  0
PetalLengthCm  0
PetalwidthCm  0
Species       0
dtype: int64

Duplicate Rows: 0

Duplicate Rows After Removal:
0

```


4. Statistical Summary

```
[5]: # Display statistical summary
print("Statistical Summary:")
print(df.describe())

# Statistics for important numeric columns
columns = [
    "SepalLengthCm",
    "SepalWidthCm",
    "PetalLengthCm",
    "PetalWidthCm"
]

for col in columns:
    print(f"\n{col} Statistics")
    print("Minimum :", df[col].min())
    print("Maximum :", df[col].max())
    print("Mean      :", df[col].mean())
    print("Median    :", df[col].median())
```

Statistical Summary:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.054000	3.758667	1.198667
std	43.445368	0.828066	0.433594	1.764420	0.763161
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000

SepalLengthCm Statistics

Minimum : 4.3
Maximum : 7.9
Mean : 5.843333333333334
Median : 5.8

SepalWidthCm Statistics

Minimum : 2.0
Maximum : 4.4
Mean : 3.0540000000000003
Median : 3.0

PetalLengthCm Statistics

Minimum : 1.0
Maximum : 6.9
Mean : 3.7586666666666666
Median : 4.35

PetalWidthCm Statistics

Minimum : 0.1
Maximum : 2.5
Mean : 1.1986666666666668
Median : 1.3

5. Histogram Analysis

```
[6]: # Numeric columns
numeric_columns = [
    "SepalLengthCm",
    "SepalWidthCm",
    "PetalLengthCm",
    "PetalWidthCm"
]

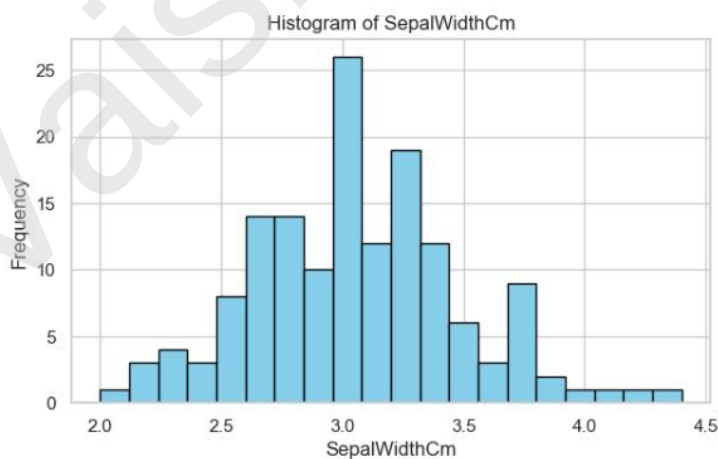
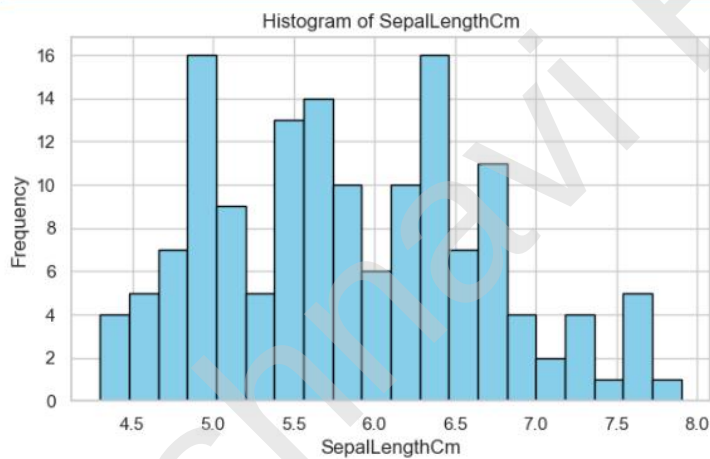
# Plot histograms
for col in numeric_columns:

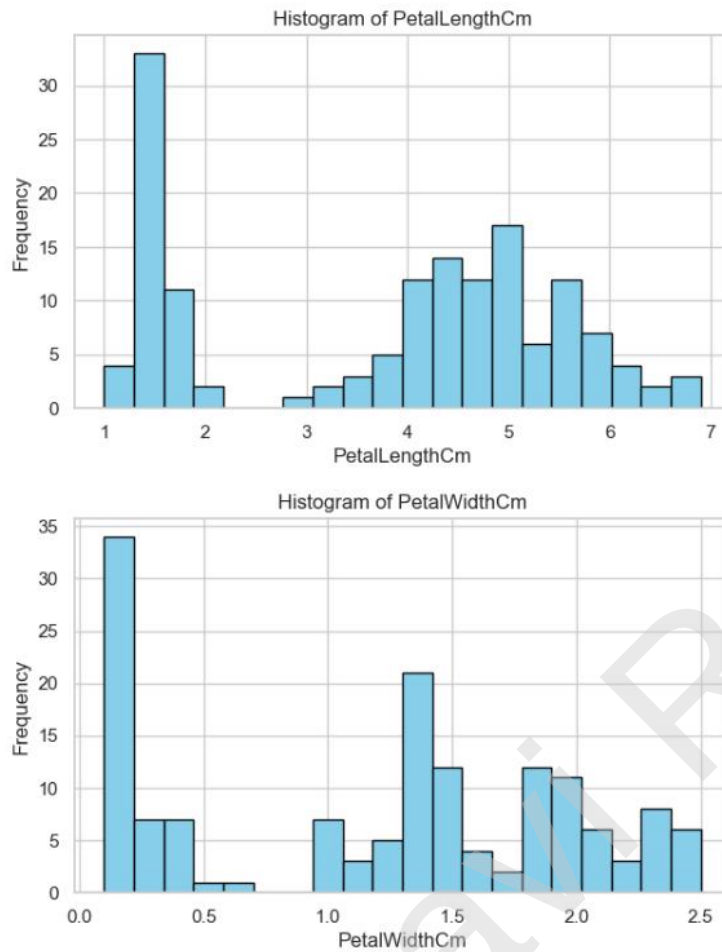
    plt.figure(figsize=(7,4))

    plt.hist(df[col], bins=20, edgecolor="black", color="skyblue")

    plt.title(f"Histogram of {col}")
    plt.xlabel(col)
    plt.ylabel("Frequency")

    plt.show()
```





6. Count Plot Analysis

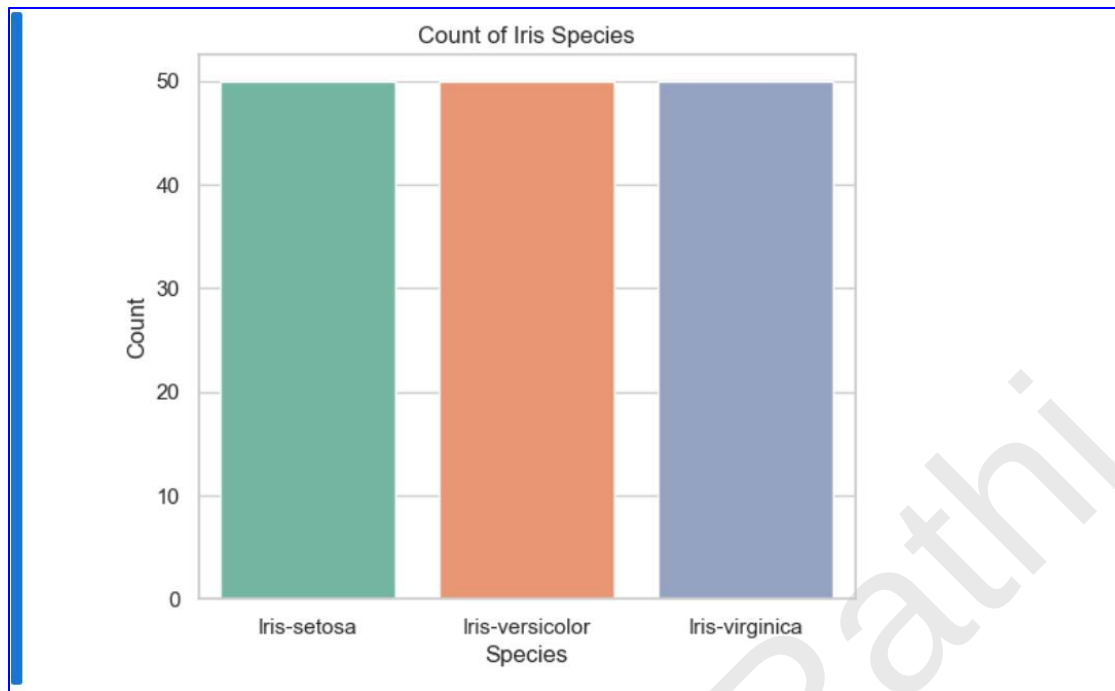
```
[7]: # Count plot for species

plt.figure(figsize=(6,5))

sns.countplot(
    data=df,
    x="Species",
    hue="Species",
    palette="Set2",
    legend=False
)

plt.title("Count of Iris Species")
plt.xlabel("Species")
plt.ylabel("Count")

plt.show()
```



7. Scatter Plot Analysis

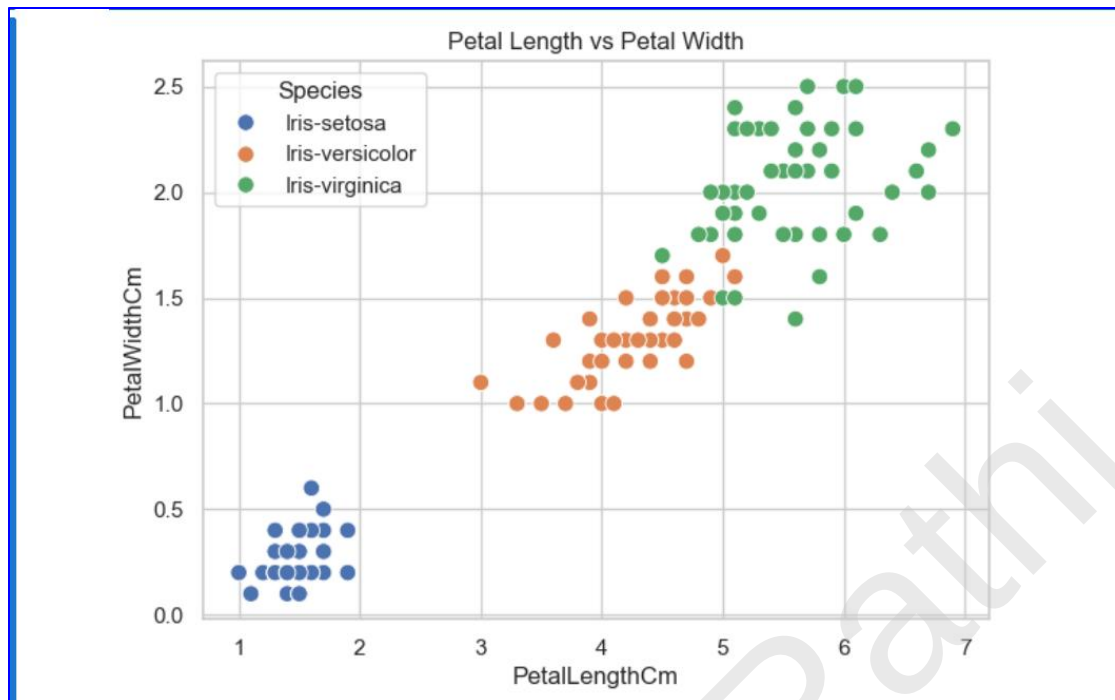
```
[8]: # Scatter plot between Petal Length and Petal Width

plt.figure(figsize=(7,5))

sns.scatterplot(
    data=df,
    x="PetalLengthCm",
    y="PetalWidthCm",
    hue="Species",
    s=70
)

plt.title("Petal Length vs Petal Width")

plt.show()
```



8. Box Plot Analysis

```
[11]: # Box plot of Sepal Length by Species

plt.figure(figsize=(7,5))

sns.boxplot(
    data=df,
    x="Species",
    y="SepalLengthCm",
    hue="Species",
    palette="Pastel1",
    legend=False
)

plt.title("Sepal Length by Species")

plt.show()
```



9. Correlation Heatmap

```
[12]: # Select numeric columns
numeric_df = df.select_dtypes(include="number")

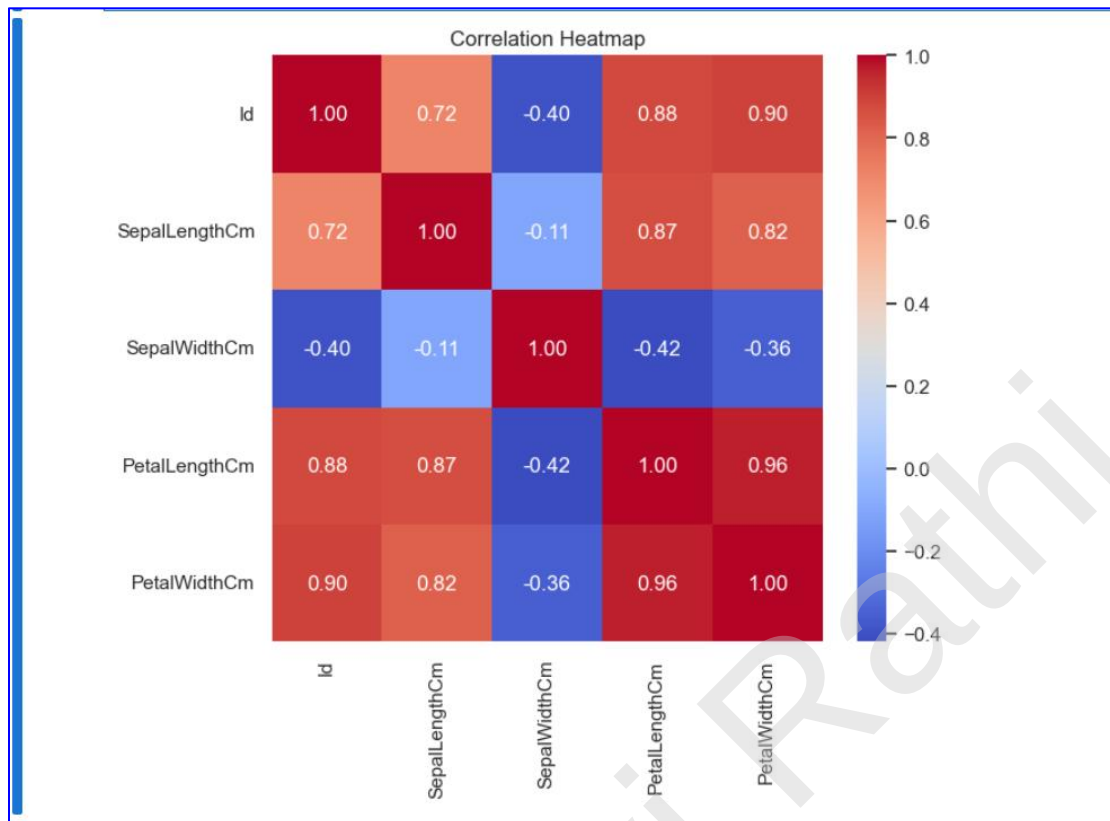
# Correlation matrix
corr_matrix = numeric_df.corr()

# Plot heatmap
plt.figure(figsize=(7,6))

sns.heatmap(
    corr_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Heatmap")

plt.show()
```



10. Feature Identification

```
[13]: # Independent features
x = df.drop("Species", axis=1)

# Remove Id column if present
if "Id" in x.columns:
    x = x.drop("Id", axis=1)

# Dependent feature
y = df["Species"]

print("Independent Features:")
print(x.columns.tolist())

print("\nDependent Feature:")
print(y.name)
```

Independent Features:
['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']

Dependent Feature:
Species

11. Label Encoding

```
[14]: # Display target values before encoding
print("Before Label Encoding:")
print(df["Species"].head())

# Apply Label Encoding
encoder = LabelEncoder()

df["Species"] = encoder.fit_transform(df["Species"])

# Display target values after encoding
print("\nAfter Label Encoding:")
print(df["Species"].head())

# Display class mapping
print("\nClass Mapping:")
for index, label in enumerate(encoder.classes_):
    print(f"{label} --> {index}")
```

```
Before Label Encoding:
0    Iris-setosa
1    Iris-setosa
2    Iris-setosa
3    Iris-setosa
4    Iris-setosa
Name: Species, dtype: object
```

```
After Label Encoding:
0    0
1    0
2    0
3    0
4    0
Name: Species, dtype: int64
```

```
Class Mapping:
Iris-setosa --> 0
Iris-versicolor --> 1
Iris-virginica --> 2
```


12. Feature Scaling

```
[15]: # Define input and output features
X = df.drop("Species", axis=1)

# Remove Id column if present
if "Id" in X.columns:
    X = X.drop("Id", axis=1)

y = df["Species"]

# Apply Standard Scaling
scaler = StandardScaler()

scaled_data = scaler.fit_transform(X)

# Convert to DataFrame
scaled_df = pd.DataFrame(
    scaled_data,
    columns=X.columns
)

# Display first 5 rows
print("First 5 Rows of Scaled Data:")
print(scaled_df.head())
```

First 5 Rows of Scaled Data:

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
0	-0.900681	1.032057	-1.341272	-1.312977
1	-1.143017	-0.124958	-1.341272	-1.312977
2	-1.385353	0.337848	-1.398138	-1.312977
3	-1.506521	0.106445	-1.284407	-1.312977
4	-1.021849	1.263460	-1.341272	-1.312977

13. Preprocessing Pipeline

```
[16]: # Remove duplicate rows
df = df.drop_duplicates()

# Define input and output features
X = df.drop("Species", axis=1)

# Remove Id column if present
if "Id" in X.columns:
    X = X.drop("Id", axis=1)

y = df["Species"]

# Encode target variable
encoder = LabelEncoder()
y = encoder.fit_transform(y)

# Scale input features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Convert scaled data to DataFrame
X_scaled = pd.DataFrame(
    X_scaled,
    columns=X.columns
)

print("Preprocessed Features:")
print(X_scaled.head())

print("\nEncoded Target:")
print(y[:5])

print("\nPreprocessing completed successfully.")
```

```
Preprocessed Features:
   SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
0      -0.900681      1.032057     -1.341272     -1.312977
1      -1.143017     -0.124958     -1.341272     -1.312977
2      -1.385353      0.337848     -1.398138     -1.312977
3      -1.506521      0.106445     -1.284407     -1.312977
4      -1.021849      1.263460     -1.341272     -1.312977

Encoded Target:
[0 0 0 0 0]

Preprocessing completed successfully.
```

14. Dataset Summary

▼ Summary

• Dataset Overview

- Explored the Iris dataset containing measurements of three Iris flower species: Setosa, Versicolor, and Virginica.
- The dataset consists of four numerical features and one target (species) column.

• Data Exploration

- Loaded and explored the dataset.
- Checked the shape, columns, and data types.

• Data Cleaning

- Checked for missing values and duplicate records.
- Verified the dataset was clean and ready for analysis.

• Data Analysis

- Generated summary statistics.
- Created histograms, count plots, scatter plots, box plots, and a correlation heatmap.
- Analyzed the distribution and relationships among the flower measurements.

• Preprocessing

- Identified the input and target features.
- Applied Label Encoding (or One Hot Encoding, if required) to the target variable.
- Applied Standard Scaling to the numerical features.

• Key Learning

- Learned the complete data preprocessing workflow for a classification dataset.
- Understood how flower measurements can be used to classify different Iris species.
- Prepared the dataset for classification machine learning models.

WineQT Dataset - Information

• Dataset Description

- The WineQT Dataset contains information about wine samples along with their physicochemical properties and quality ratings.
- It includes chemical characteristics of wine that can affect its overall quality.
- The dataset is useful for analyzing patterns and building machine learning models.

• Features

- Fixed Acidity
- Volatile Acidity
- Citric Acid
- Residual Sugar
- Chlorides
- Free Sulfur Dioxide
- Total Sulfur Dioxide
- Density
- pH
- Sulphates
- Alcohol
- Other chemical properties

• Target Variable

- Quality

• Machine Learning Type

- Regression / Classification

• Purpose

- To analyze the factors influencing wine quality.
- To predict wine quality based on physicochemical properties.

Program Output:

1. Import Libraries

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler, LabelEncoder

plt.style.use("ggplot")
sns.set_theme(style="whitegrid")
```

2. Data Loading

```
[2]: # Load the dataset
df = pd.read_csv("WineQT.csv")

# Display first 10 rows
print("First 10 Rows:")
print(df.head(10))

# Display last 5 rows
print("\nLast 5 Rows:")
print(df.tail())

# Display dataset shape
print("\nDataset Shape:")
print(df.shape)

# Display column names
print("\nColumn Names:")
print(df.columns.tolist())

# Display data types
print("\nData Types:")
print(df.dtypes)

# Display dataset information
print("\nDataset Information:")
df.info()
```

First 10 Rows:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides \
0	7.4	0.70	0.00	1.9	0.076
1	7.8	0.88	0.00	2.6	0.098
2	7.8	0.76	0.04	2.3	0.092
3	11.2	0.28	0.56	1.9	0.075
4	7.4	0.70	0.00	1.9	0.076
5	7.4	0.66	0.00	1.8	0.075
6	7.9	0.60	0.06	1.6	0.069
7	7.3	0.65	0.00	1.2	0.065
8	7.8	0.58	0.02	2.0	0.073
9	6.7	0.58	0.08	1.8	0.097

	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates \
0	11.0	34.0	0.9978	3.51	0.56
1	25.0	67.0	0.9968	3.20	0.68
2	15.0	54.0	0.9970	3.26	0.65
3	17.0	60.0	0.9980	3.16	0.58
4	11.0	34.0	0.9978	3.51	0.56
5	13.0	40.0	0.9978	3.51	0.56
6	15.0	59.0	0.9964	3.30	0.46
7	15.0	21.0	0.9946	3.39	0.47
8	9.0	18.0	0.9968	3.36	0.57
9	15.0	65.0	0.9959	3.28	0.54

	alcohol	quality	Id
0	9.4	5	0
1	9.8	5	1
2	9.8	5	2
3	9.8	6	3
4	9.4	5	4
5	9.4	5	5
6	9.4	5	6
7	10.0	7	7
8	9.5	7	8
9	9.2	5	10

Last 5 Rows:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides \
1138	6.3	0.510	0.13	2.3	0.076
1139	6.8	0.620	0.08	1.9	0.068
1140	6.2	0.600	0.08	2.0	0.090
1141	5.9	0.550	0.10	2.2	0.062
1142	5.9	0.645	0.12	2.0	0.075

	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates \
1138	29.0	40.0	0.99574	3.42	0.75
1139	28.0	38.0	0.99651	3.42	0.82
1140	32.0	44.0	0.99490	3.45	0.58
1141	39.0	51.0	0.99512	3.52	0.76
1142	32.0	44.0	0.99547	3.57	0.71

	alcohol	quality	Id
1138	11.0	6	1592
1139	9.5	6	1593
1140	10.5	5	1594
1141	11.2	6	1595
1142	10.2	5	1597

Dataset Shape:
(1143, 13)

Column Names:
['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar', 'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'density', 'pH', 'sulphates', 'alcohol', 'quality', 'Id']

Data Types:

fixed acidity	float64
volatile acidity	float64
citric acid	float64
residual sugar	float64
chlorides	float64
free sulfur dioxide	float64
total sulfur dioxide	float64
density	float64
pH	float64
sulphates	float64
alcohol	float64
quality	int64
Id	int64
dtype:	object

```

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1143 entries, 0 to 1142
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  ---                ---
0   fixed acidity          1143 non-null   float64
1   volatile acidity       1143 non-null   float64
2   citric acid            1143 non-null   float64
3   residual sugar         1143 non-null   float64
4   chlorides              1143 non-null   float64
5   free sulfur dioxide    1143 non-null   float64
6   total sulfur dioxide   1143 non-null   float64
7   density                1143 non-null   float64
8   pH                    1143 non-null   float64
9   sulphates              1143 non-null   float64
10  alcohol                1143 non-null   float64
11  quality                1143 non-null   int64
12  Id                     1143 non-null   int64
dtypes: float64(11), int64(2)
memory usage: 116.2 KB

```

3. Missing Values

```

[3]: # Check missing values
print("Missing Values:")
print(df.isnull().sum())

# Check duplicate rows
duplicate_count = df.duplicated().sum()

print("\nDuplicate Rows:", duplicate_count)

# Remove duplicate rows
df = df.drop_duplicates()

print("\nDuplicate Rows After Removal:")
print(df.duplicated().sum())

```

```

Missing Values:
fixed acidity          0
volatile acidity       0
citric acid            0
residual sugar         0
chlorides              0
free sulfur dioxide    0
total sulfur dioxide   0
density                0
pH                    0
sulphates              0
alcohol                0
quality                0
Id                     0
dtype: int64

```

```
Duplicate Rows: 0
```

```
Duplicate Rows After Removal:
0

```


4. Statistical Summary

```
[4]: # Display statistical summary
print("Statistical Summary:")
print(df.describe())

columns = [
    "fixed acidity",
    "volatile acidity",
    "citric acid",
    "alcohol",
    "quality"
]

for col in columns:
    print(f"\n{col.upper()} Statistics")
    print("Minimum :", df[col].min())
    print("Maximum :", df[col].max())
    print("Mean      :", df[col].mean())
    print("Median   :", df[col].median())
```

```
Statistical Summary:
      fixed acidity  volatile acidity  citric acid  residual sugar \
count    1143.000000      1143.000000    1143.000000    1143.000000
mean       8.311111       0.531339     0.268364       2.532152
std       1.747595       0.179633     0.196686       1.355917
min        4.600000       0.120000     0.000000       0.900000
25%        7.100000       0.392500     0.090000       1.900000
50%        7.900000       0.520000     0.250000       2.200000
75%        9.100000       0.640000     0.420000       2.600000
max       15.900000       1.580000     1.000000      15.500000

      chlorides  free sulfur dioxide  total sulfur dioxide  density \
count    1143.000000      1143.000000      1143.000000    1143.000000
mean       0.086933      15.615486      45.914698     0.996730
std       0.047267      10.250486      32.782130     0.001925
min       0.012000       1.000000       6.000000     0.990070
25%       0.070000       7.000000      21.000000     0.995570
50%       0.079000      13.000000      37.000000     0.996680
75%       0.090000      21.000000      61.000000     0.997845
max       0.611000      68.000000     289.000000     1.003690

      pH  sulphates  alcohol  quality  Id
count    1143.000000    1143.000000    1143.000000    1143.000000    1143.000000
mean       3.311015     0.657708     10.442111     5.657043    804.969379
std       0.156664     0.170399     1.082196     0.805824    463.997116
min       2.740000     0.330000     8.400000     3.000000     0.000000
25%       3.205000     0.550000     9.500000     5.000000    411.000000
50%       3.310000     0.620000    10.200000     6.000000    794.000000
75%       3.400000     0.730000    11.100000     6.000000   1209.500000
max       4.010000     2.000000    14.900000     8.000000   1597.000000
```



```
FIXED ACIDITY Statistics
Minimum : 4.6
Maximum : 15.9
Mean    : 8.311111111111112
Median  : 7.9

VOLATILE ACIDITY Statistics
Minimum : 0.12
Maximum : 1.58
Mean    : 0.5313385826771653
Median  : 0.52

CITRIC ACID Statistics
Minimum : 0.0
Maximum : 1.0
Mean    : 0.2683639545056868
Median  : 0.25

ALCOHOL Statistics
Minimum : 8.4
Maximum : 14.9
Mean    : 10.442111402741325
Median  : 10.2

QUALITY Statistics
Minimum : 3
Maximum : 8
Mean    : 5.657042869641295
Median  : 6.0
```

5. Histogram Analysis

```
[5]: numeric_columns = [
      "fixed acidity",
      "volatile acidity",
      "citric acid",
      "alcohol",
      "quality"
    ]

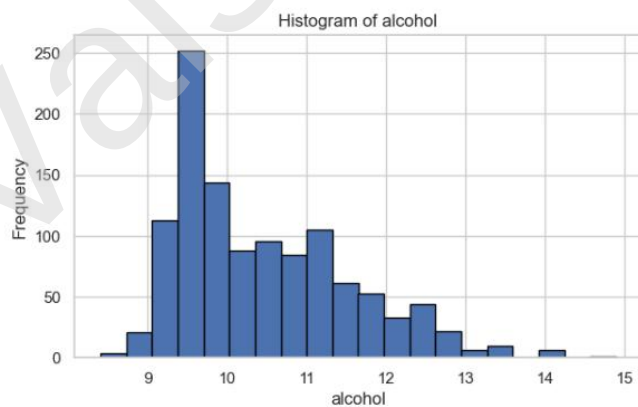
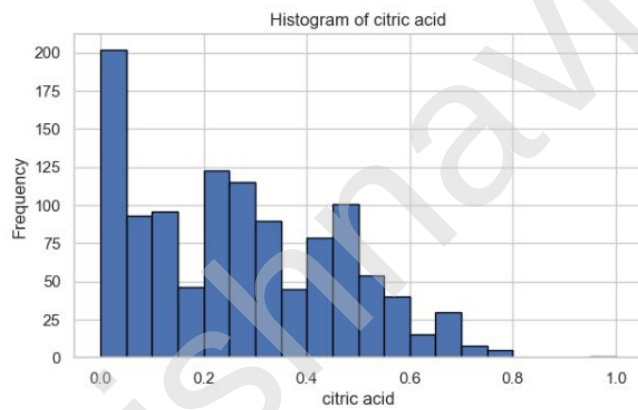
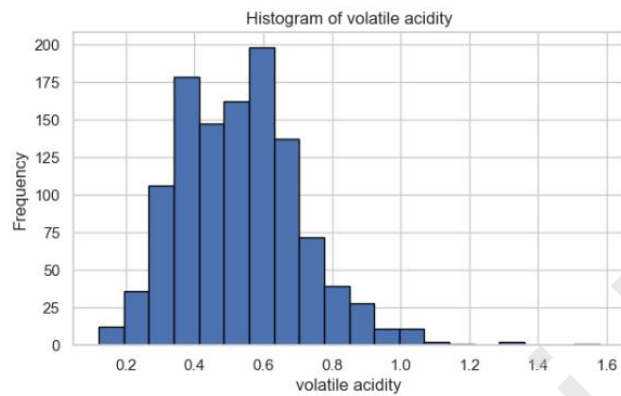
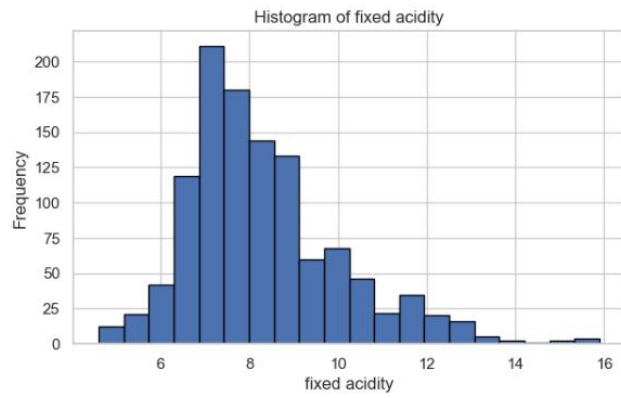
    for col in numeric_columns:

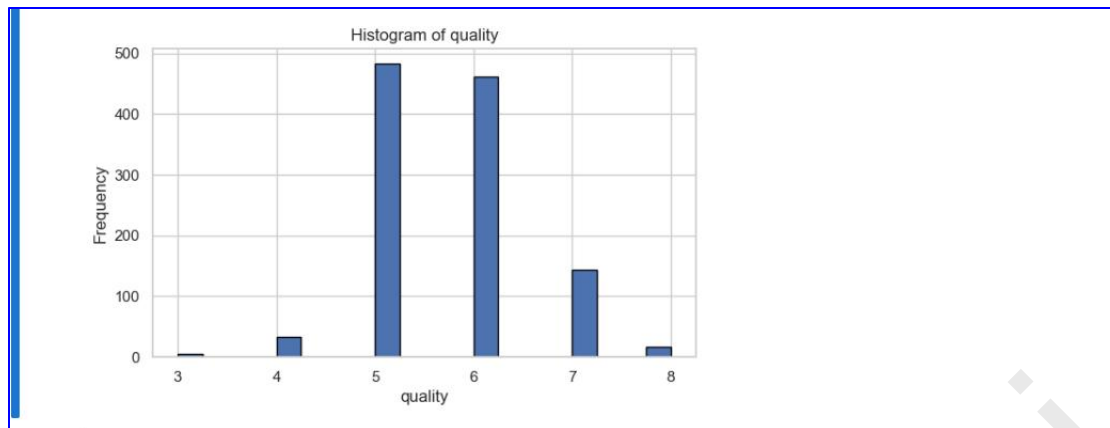
        plt.figure(figsize=(7,4))

        plt.hist(df[col], bins=20, edgecolor="black")

        plt.title(f"Histogram of {col}")
        plt.xlabel(col)
        plt.ylabel("Frequency")

        plt.show()
```



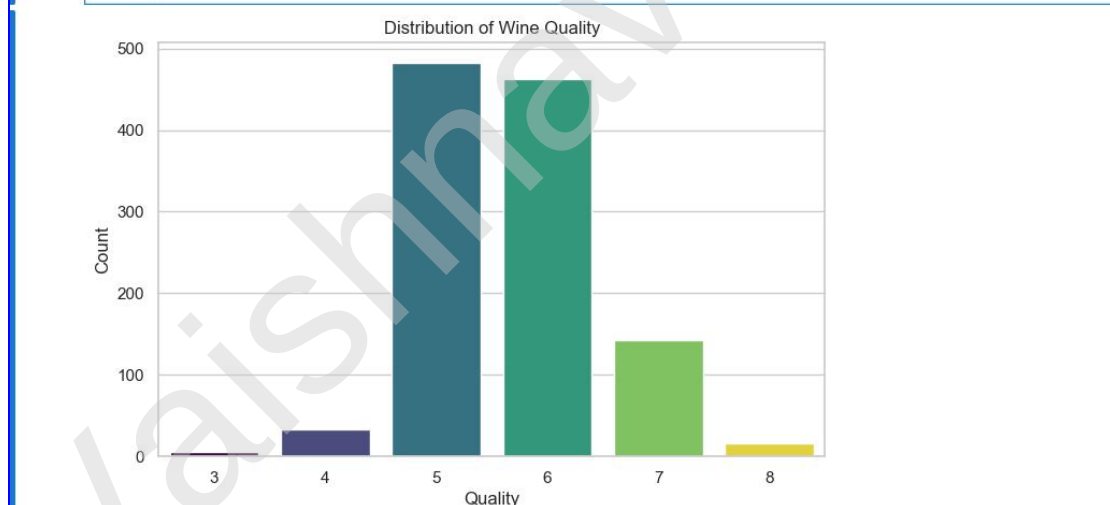


6. Count Plot Analysis

```
[6]: # Count plot of wine quality
plt.figure(figsize=(8,5))

sns.countplot(
    data=df,
    x="quality",
    hue="quality",
    palette="viridis",
    legend=False
)

plt.title("Distribution of Wine Quality")
plt.xlabel("Quality")
plt.ylabel("Count")
plt.show()
```



7. Scatter Plot Analysis

```
[7]: # Scatter plot between Alcohol and Quality
```

```
plt.figure(figsize=(7,5))
```

```
sns.scatterplot(
```

```
    data=df,
```

```
    x="alcohol",
```

```
    y="quality",
```

```
    alpha=0.7
```

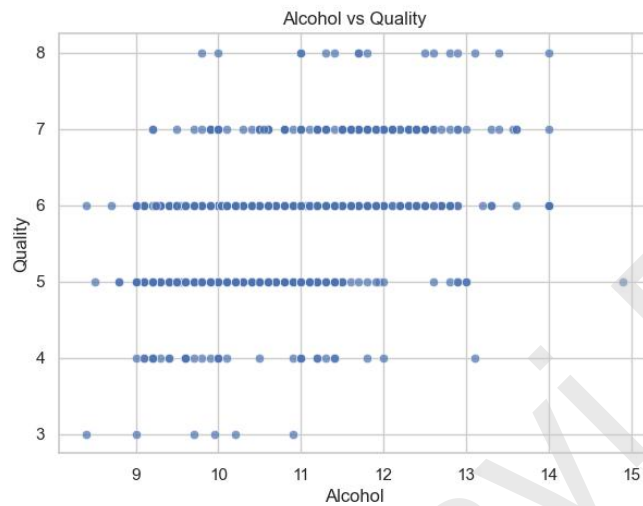
```
)
```

```
plt.title("Alcohol vs Quality")
```

```
plt.xlabel("Alcohol")
```

```
plt.ylabel("Quality")
```

```
plt.show()
```

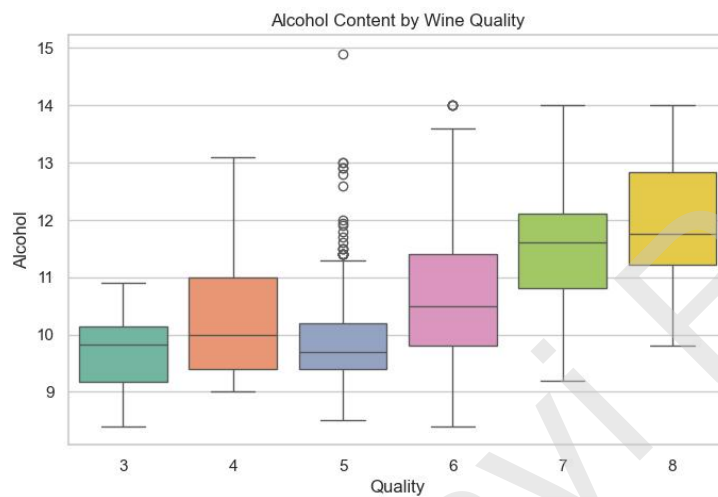


8. Box Plot Analysis

```
[13]: # Box plot of Alcohol by Quality
plt.figure(figsize=(8,5))

sns.boxplot(
    data=df,
    x="quality",
    y="alcohol",
    hue="quality",
    palette="Set2",
    legend=False
)

plt.title("Alcohol Content by Wine Quality")
plt.xlabel("Quality")
plt.ylabel("Alcohol")
plt.show()
```



9. Correlation Heatmap

```
[9]: # Select numeric columns
numeric_df = df.select_dtypes(include="number")

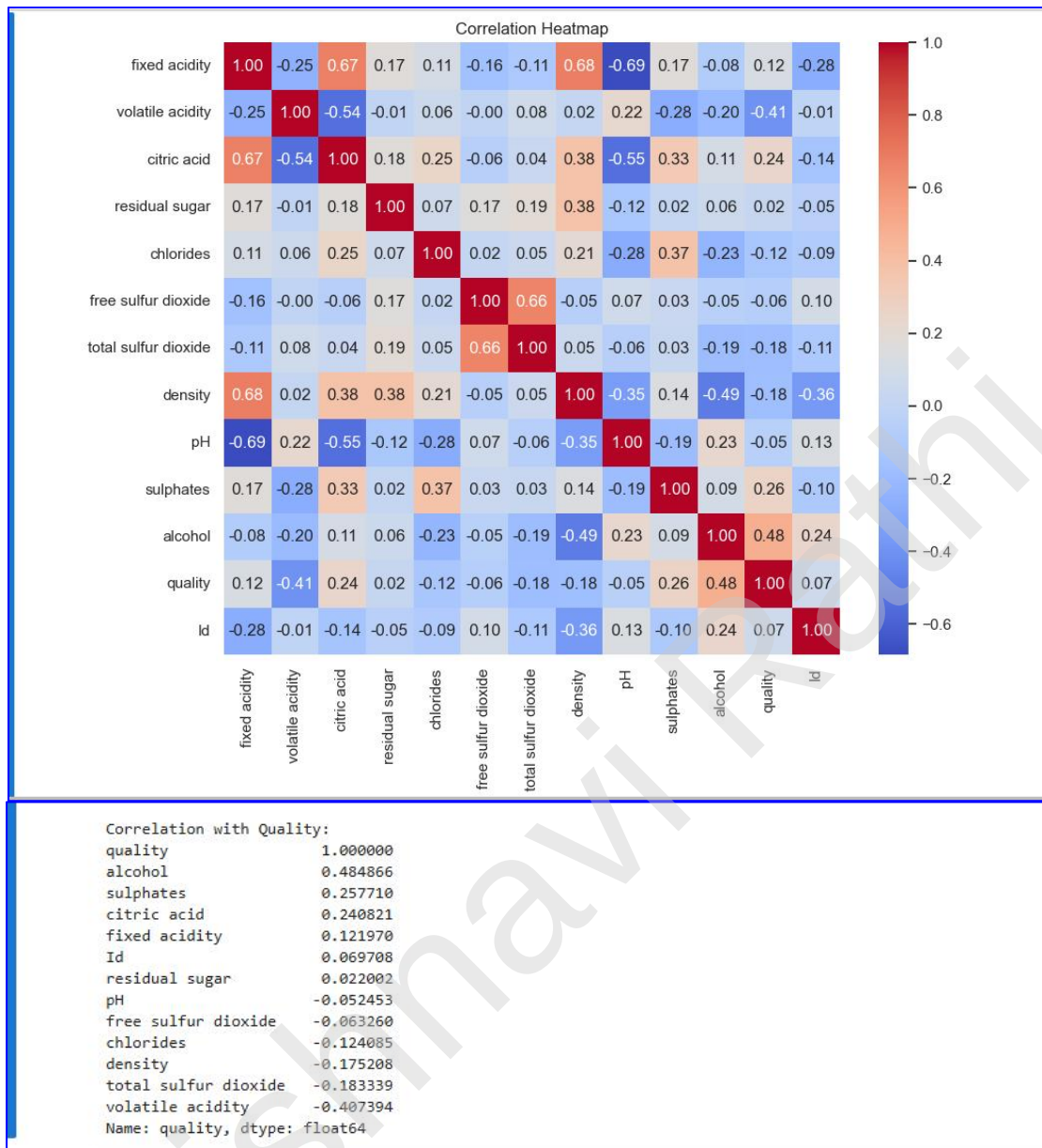
# Correlation matrix
corr_matrix = numeric_df.corr()

# Plot heatmap
plt.figure(figsize=(10,8))

sns.heatmap(
    corr_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Heatmap")
plt.show()

# Correlation with target variable
print("Correlation with Quality:")
print(corr_matrix["quality"].sort_values(ascending=False))
```



10. Feature Identification

```

[10]: # Independent Features
X = df.drop("quality", axis=1)

# Remove Id column if present
if "Id" in X.columns:
    X = X.drop("Id", axis=1)

# Dependent Feature
y = df["quality"]

print("Independent Features:")
print(X.columns.tolist())

print("\nDependent Feature:")
print(y.name)

Independent Features:
['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar', 'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'density', 'pH', 'sulphates', 'alcohol']

Dependent Feature:
quality

```

11. Feature Scaling

```
[11]: from sklearn.preprocessing import StandardScaler

# Apply Standard Scaling
scaler = StandardScaler()

scaled_data = scaler.fit_transform(X)

# Convert scaled data to DataFrame
scaled_df = pd.DataFrame(
    scaled_data,
    columns=X.columns
)

print("First 5 Rows of Scaled Data:")
print(scaled_df.head())
```

First 5 Rows of Scaled Data:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	\
0	-0.521580	0.939332	-1.365027	-0.466421	-0.231395	
1	-0.292593	1.941813	-1.365027	0.050060	0.234247	
2	-0.292593	1.273492	-1.161568	-0.171289	0.107253	
3	1.653789	-1.399789	1.483400	-0.466421	-0.252560	
4	-0.521580	0.939332	-1.365027	-0.466421	-0.231395	

	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	\
0	-0.450467	-0.363610	0.555854	1.270695	-0.573658	
1	0.915920	0.643477	0.036165	-0.708928	0.130881	
2	-0.060071	0.246745	0.140103	-0.325775	-0.045254	
3	0.135127	0.429852	0.659792	-0.964363	-0.456235	
4	-0.450467	-0.363610	0.555854	1.270695	-0.573658	

	alcohol
0	-0.963382
1	-0.593601
2	-0.593601
3	-0.593601
4	-0.963382

12. Preprocessing Pipeline

```
[12]: from sklearn.preprocessing import StandardScaler
```

```
# Remove duplicate rows
df = df.drop_duplicates()

# Define input and output features
X = df.drop("quality", axis=1)

# Remove Id column if present
if "Id" in X.columns:
    X = X.drop("Id", axis=1)

y = df["quality"]

# Apply Standard Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Convert scaled data to DataFrame
X_scaled = pd.DataFrame(
    X_scaled,
    columns=X.columns
)

print("Preprocessed Features:")
print(X_scaled.head())

print("\nTarget Variable:")
print(y.head())

print("\nPreprocessing completed successfully.")
```

```
Preprocessed Features:
  fixed acidity  volatile acidity  citric acid  residual sugar  chlorides \
0   -0.521580         0.939332   -1.365027   -0.466421   -0.231395
1   -0.292593         1.941813   -1.365027    0.050060    0.234247
2   -0.292593         1.273492   -1.161568   -0.171289    0.107253
3    1.653789        -1.399789    1.483400   -0.466421   -0.252560
4   -0.521580         0.939332   -1.365027   -0.466421   -0.231395

  free sulfur dioxide  total sulfur dioxide  density      pH  sulphates \
0   -0.450467        -0.363610  0.555854  1.270695  -0.573658
1    0.915920         0.643477  0.036165 -0.708928   0.130881
2   -0.060071         0.246745  0.140103 -0.325775  -0.045254
3    0.135127         0.429852  0.659792 -0.964363  -0.456235
4   -0.450467        -0.363610  0.555854  1.270695  -0.573658

  alcohol
0 -0.963382
1 -0.593601
2 -0.593601
3 -0.593601
4 -0.963382

Target Variable:
0    5
1    5
2    5
3    6
4    5
Name: quality, dtype: int64

Preprocessing completed successfully.
```


13. Dataset Summary

Summary

- Dataset Overview**
 - Explored the WineQT dataset containing various physicochemical properties of wine and their corresponding quality ratings.
 - The dataset consists mainly of numerical features used to evaluate wine quality.
- Data Exploration**
 - Loaded and explored the dataset.
 - Checked the shape, columns, and data types.
- Data Cleaning**
 - Checked for missing values and duplicate records.
 - Verified the dataset was clean and ready for analysis.
- Data Analysis**
 - Generated summary statistics.
 - Created histograms, a count plot, a scatter plot, a box plot, and a correlation heatmap.
 - Analyzed the relationship between different wine properties and quality.
- Preprocessing**
 - Identified the input and target features.
 - Applied Standard Scaling to the numerical features.
- Key Learning**
 - Learned the complete data preprocessing workflow for a wine quality dataset.
 - Understood how different physicochemical properties influence wine quality.
 - Prepared the dataset for machine learning analysis.

Note

The WineQT dataset contains only numerical features (apart from an optional **Id** column), so **One Hot Encoding** or **Label Encoding** is not required. The target variable is **quality**, which is already numeric.